Assignment

Medical Supply Inventory Management System

GenAI Level 1 — Limited AI Use ⚠️

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the **Generative AI Use in Assessments policy** posted at the top of the **Assignments** section in Moodle.

---

1. Scenario

A community health clinic maintains an inventory of medical supplies used in its daily operations, including examination gloves, masks, bandages, disinfectants, thermometers, and other clinical supplies.

Currently, employees maintain inventory information manually. The clinic would like a simple web-based system that allows staff to manage its medical supply inventory.

Your task is to develop a **Spring Boot CRUD web application** that allows users to:

- view all medical supplies;
- add a new medical supply;
- update an existing medical supply;
- delete a medical supply.

All information must be stored persistently in a **MySQL database**, and the application must provide a browser-based interface using **Thymeleaf**.

---

2. Objective

Develop a complete database-backed web application using:

- Spring Boot;
- Spring Web;
- Spring Data JPA;
- MySQL;
- Thymeleaf;
- HTML/CSS/JavaScript as needed.

Your application must implement all four CRUD operations: **Create → Read → Update → Delete**

3. Application Model

The application will contain **one model/entity**:

MedicalSupply

Each medical supply must contain the following information:

Field	Description
`id`	Unique identifier
`name`	Name of the medical supply
`category`	Supply category
`vendor`	Name of the vendor
`unitPrice`	Price per unit
`quantity`	Quantity currently in stock
`description`	Short description

Example data:

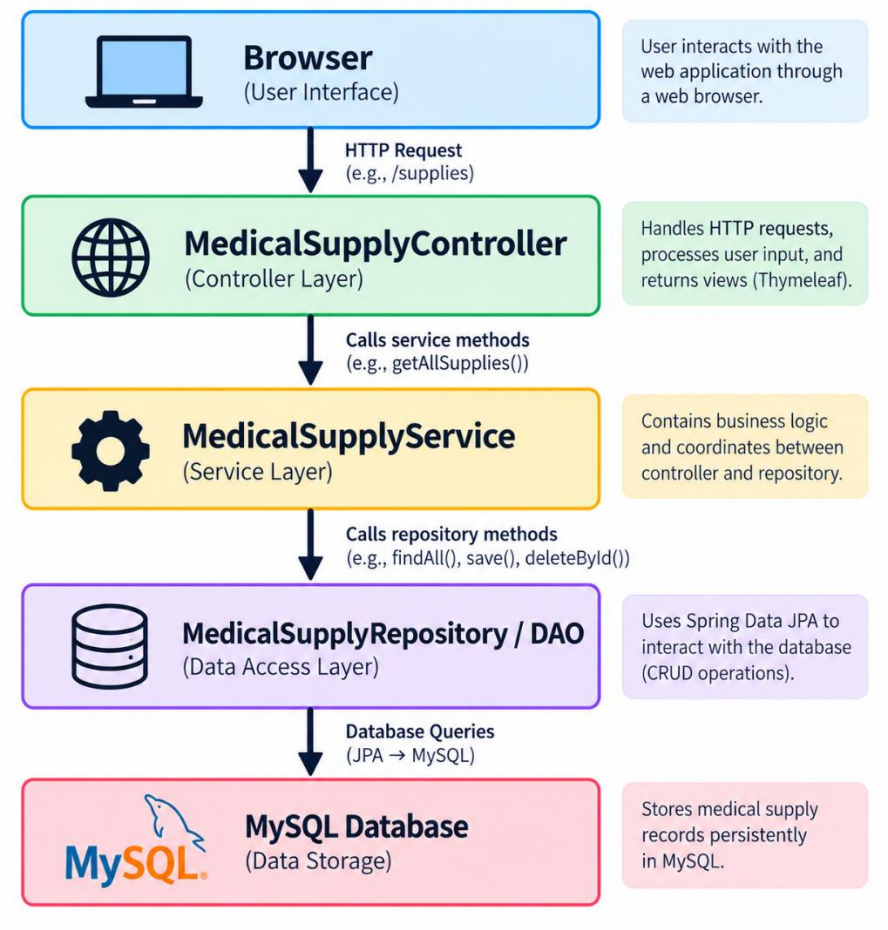
- **`ID`**: 1
- **`Name`**: Examination Gloves
- **`Category`**: Personal Protective Equipment
- **`Vendor`**: MedSupply Canada
- **`Unit Price`**: 12.99
- **`Quantity`**: 50
- **`Description`**: Disposable examination gloves

`category` and **`vendor`** are simply fields of the **`MedicalSupply`** entity. **No additional tables or entity relationships are required.**

---

4. Application Architecture

Your application must follow a layered architecture:

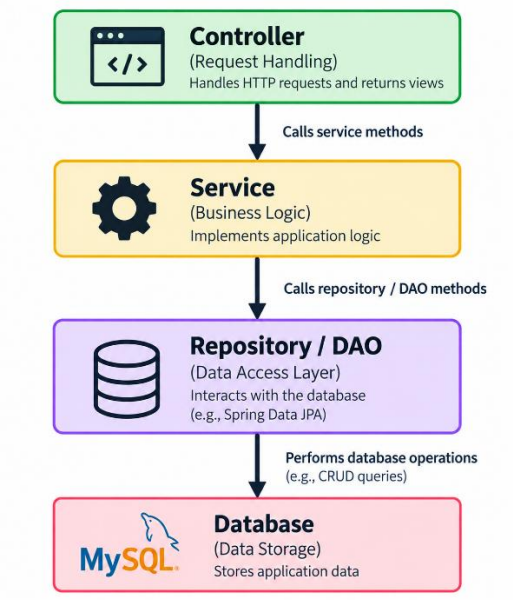


Each layer has a specific responsibility:

- **Controller** — handles browser requests and navigation.
- **Service** — provides the application's service/business layer.
- **Repository/DAO** — communicates with the database.
- **Database** — persistently stores medical supply records.

Important

Do not place all application logic directly inside the controller. Maintain a clear **Controller → Service → Repository/DAO → Database** structure.



---

5. Part A — Create the Spring Boot Project

Create a Spring Boot project using **Spring Initializr**.

Add the necessary dependencies for:

- Spring Web;
- Thymeleaf;
- Spring Data JPA;
- MySQL Driver.

Import the generated project into your IDE and verify that the application starts successfully.

---

6. Part B — Create the MySQL Database

Using **MySQL Workbench**, create a new schema for the application.

Create a table for storing medical supplies.

The table must contain appropriate columns corresponding to:

- id
- name
- category

- vendor
- unit_price
- quantity
- description

Configure your Spring Boot application to connect successfully to the MySQL database.

Practical Requirement

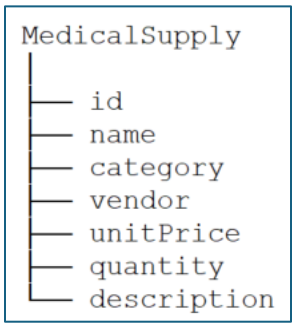
The application must use the MySQL database for persistent storage. Changes made through the web application must be reflected in the database.

---

7. Part C — Create the MedicalSupply Entity

Create a Java entity representing a medical supply.

The entity should map appropriately to the corresponding MySQL table and contain the required fields:



Use appropriate JPA annotations to identify the class as an entity and configure its primary key.

---

8. Part D — Create the Repository / DAO Layer

Create a repository for the **`MedicalSupply`** entity using **Spring Data JPA**.

The repository must support the database operations required for CRUD functionality, including operations conceptually equivalent to:

- **`findAll()`**
- **`findById()`**
- **`save()`**
- **`deleteById()`**

Spring Data JPA should be used to perform these persistence operations rather than manually writing SQL for every CRUD operation.

---

9. Part E — Create the Service Layer

Create a service layer between the controller and repository.

It must provide functionality to:

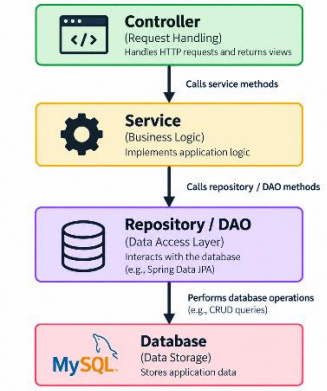
`Retrieve all medical supplies`

`Retrieve one medical supply by ID`

`Save a medical supply`

`Delete a medical supply by ID`

The request flow should therefore remain:



---

10. Part F — READ: Display Medical Supplies

Create a Thymeleaf page that displays all medical supplies stored in the database.

For example:

`Medical Supply Inventory`

```
Medical Supply Inventory
-------------------------------------------------------------------------
ID   Name            Category        Vendor        Price   Qty   Action
-------------------------------------------------------------------------
1    Exam Gloves     PPE             MedSupply     12.99   50    Update Delete
2    Bandages        First Aid       HealthPro      8.50   25    Update Delete
3    Disinfectant    Cleaning        CleanCare      6.75   40    Update Delete

[ Add Medical Supply ]
```

The information displayed on this page must be retrieved from **MySQL**, not hard-coded into the HTML page or controller.

---

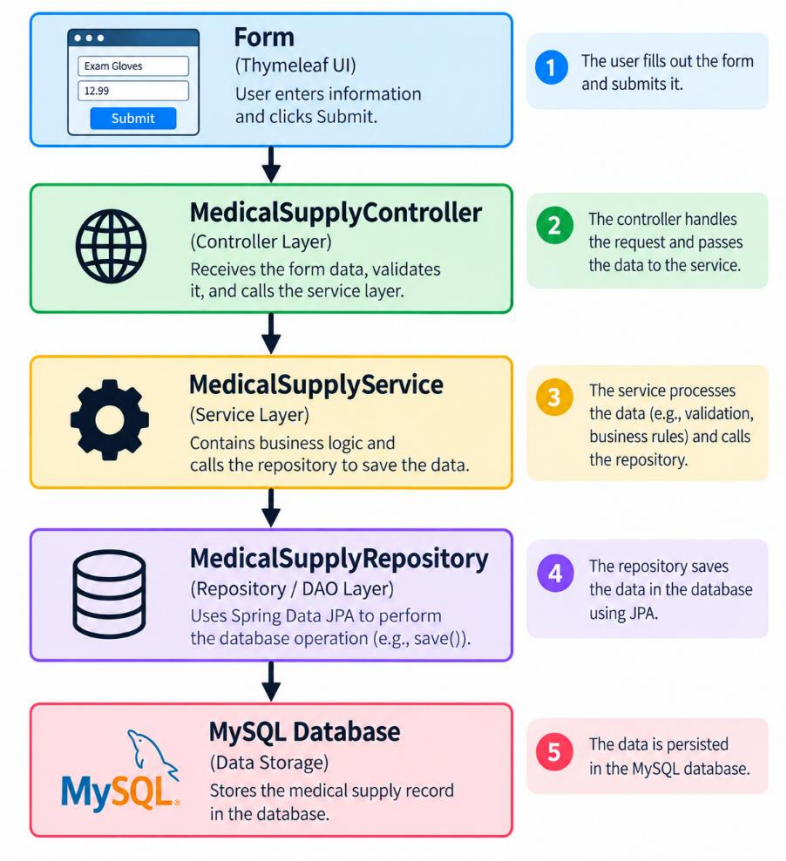
11. Part G — CREATE: Add a Medical Supply

Add an **Add Medical Supply** button to the inventory page.

The button should open a Thymeleaf form allowing the user to enter:

- Name
- Category
- Vendor
- Unit Price
- Quantity
- Description

When the form is submitted:



The new record must then appear in the medical supply list.

---

12. Part H — UPDATE: Modify a Medical Supply

Add an **Update** action for every medical supply.

When the user selects Update:

1. identify the selected medical supply using its ID;
2. retrieve it from the database;
3. display its current information in a pre-populated form;
4. allow the user to modify the information;
5. save the changes to MySQL;
6. return the user to the updated inventory list.

The existing ID must be preserved so that the application updates the existing record instead of creating an unrelated new record.

---

13. Part I — DELETE: Remove a Medical Supply

Add a **Delete** action for each medical supply.

When the user selects Delete, the application must remove the corresponding record from the database.

A confirmation message should be displayed before deletion, for example:

Are you sure you want to delete this medical supply?

After successful deletion, redirect the user to the updated inventory list.

---

14. Part J — Thymeleaf User Interface

Use **Thymeleaf** to create the browser interface.

At minimum, your application should contain:

- an inventory/list page;
- an Add Medical Supply form;
- an Update Medical Supply form or a reusable Add/Update form;
- Update and Delete controls.

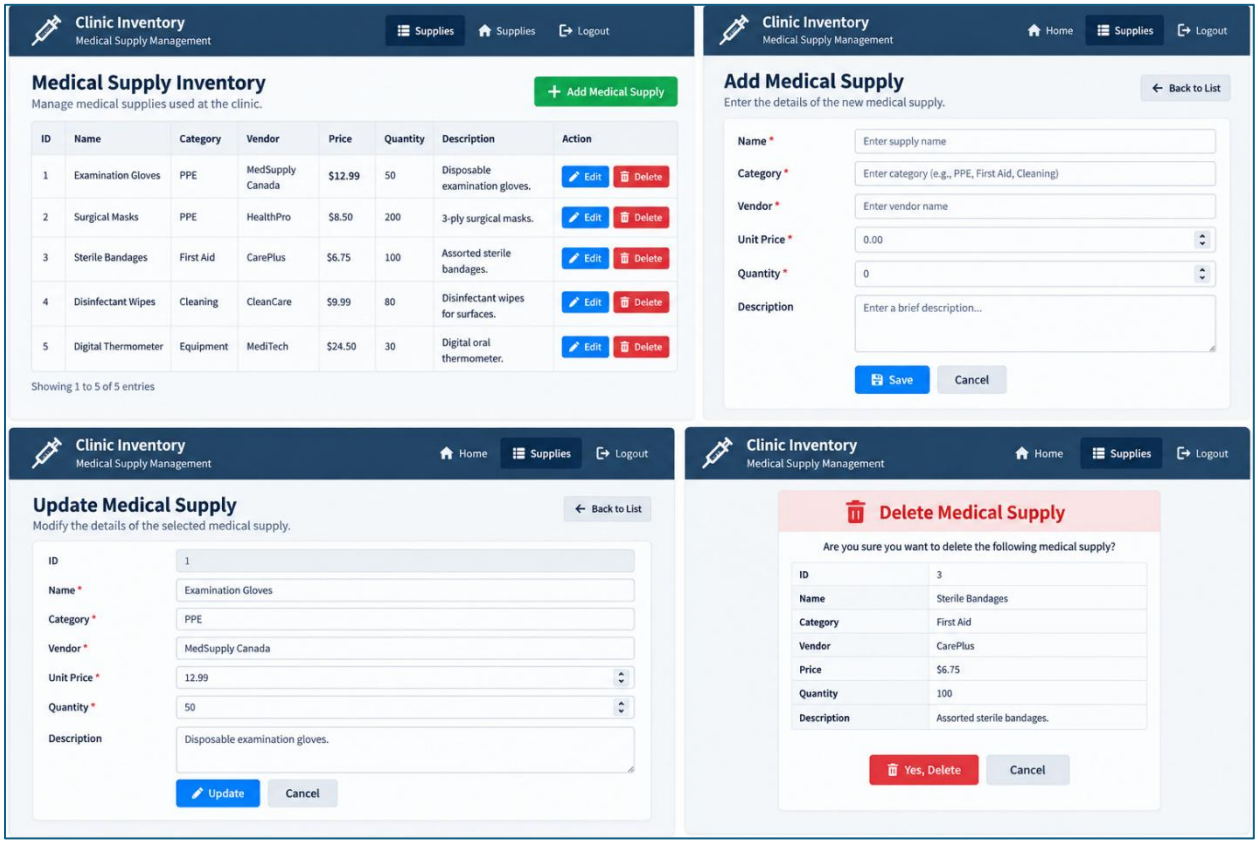
Use Thymeleaf data binding to connect the HTML forms to your Java model.

You may use **Bootstrap, CSS, and JavaScript** to improve the interface.

Tip

Focus primarily on correct functionality and a clean, usable interface. An elaborate visual design is not required.

Expected GUI: The following example illustrates the expected user interface for managing medical supplies and performing the required CRUD operations.



---

15. Part K — Validation and Error Handling

Add appropriate validation to prevent obviously invalid inventory information.

For example:

- name should not be empty;
- category should not be empty;
- vendor should not be empty;
- price should contain an appropriate numeric value;
- quantity should contain an appropriate numeric value.

The application should handle invalid input appropriately instead of producing an uncontrolled application error.

---

16. Practical Testing Requirements

You must demonstrate all four CRUD operations.

Operation	Required Demonstration
Create	Add a new medical supply and verify it in MySQL
Read	Display database records in the Thymeleaf inventory page
Update	Modify an existing supply and verify the change in MySQL
Delete	Delete a supply and verify its removal from MySQL

Students should use **MySQL Workbench** to confirm that operations performed through the web application actually modify the database.

---

17. Practical Report and Screenshot Requirements

This is a **practical programming assignment**. Your report must clearly demonstrate that you designed, implemented, ran, and tested the application.

You must include a **comprehensive set of screenshots** demonstrating the practical requirements.

The screenshots should include evidence of:

- Spring Initializr/project configuration;
- project structure;
- required Maven dependencies;
- MySQL schema and medical_supply table;
- database connection configuration;
- MedicalSupply entity;
- Repository/DAO;
- Service;
- Controller;
- Thymeleaf templates;
- running inventory page;

- **Create** operation;
- **Read** operation;
- **Update** operation;
- **Delete** operation;
- MySQL Workbench evidence showing database changes.

Each screenshot should be clear and readable and include a short explanation of what it demonstrates.

Important — Practical Evidence

Source-code screenshots alone are **not sufficient**. The report must demonstrate the **running application and successful CRUD operations**, including evidence that the corresponding data is actually stored, modified, and deleted in MySQL.

---

18. Deliverables

Submit the following through **Moodle**:

1. Complete Spring Boot project/source code.
2. Practical report explaining the application's implementation.
3. Comprehensive set of screenshots demonstrating all requirements and CRUD operations.

---

19. Due Date

You must respect the **due date specified on Moodle**.

It is your responsibility to verify the deadline and make sure that all required files have been successfully submitted before the Moodle deadline.

---

20. Mandatory Presentation to the Professor

In addition to submitting your assignment on Moodle, you must **present and demonstrate the working application to the professor to receive a grade**.

During the demonstration, the professor may ask you to perform a Create, Read, Update, or Delete operation, verify the corresponding record directly in MySQL, explain the **Controller → Service → Repository/DAO → Database** architecture, or explain selected parts of your implementation.

You should therefore understand your code and be able to explain how data travels through the different application layers.

Mandatory Grading Condition

Submission on Moodle alone is not sufficient. The assignment must also be presented to the professor. If the assignment is not presented, it will not be graded.

Rubric

Criterion	Description	Weight
Spring Boot & MySQL Setup	Correct project, dependencies, database, entity, and connection configuration	15%
Application Architecture	Correct Controller, Service, and Repository/DAO separation	15%
CRUD Functionality	Create, Read, Update, and Delete operate correctly with MySQL	30%
Thymeleaf UI & Validation	Functional forms, inventory table, navigation, and appropriate validation	15%
Practical Report & Screenshots	Comprehensive evidence of implementation, CRUD testing, and database changes	15%
Live Demonstration & Understanding	Successful presentation and understanding of the implementation	10%
Total		**100%**